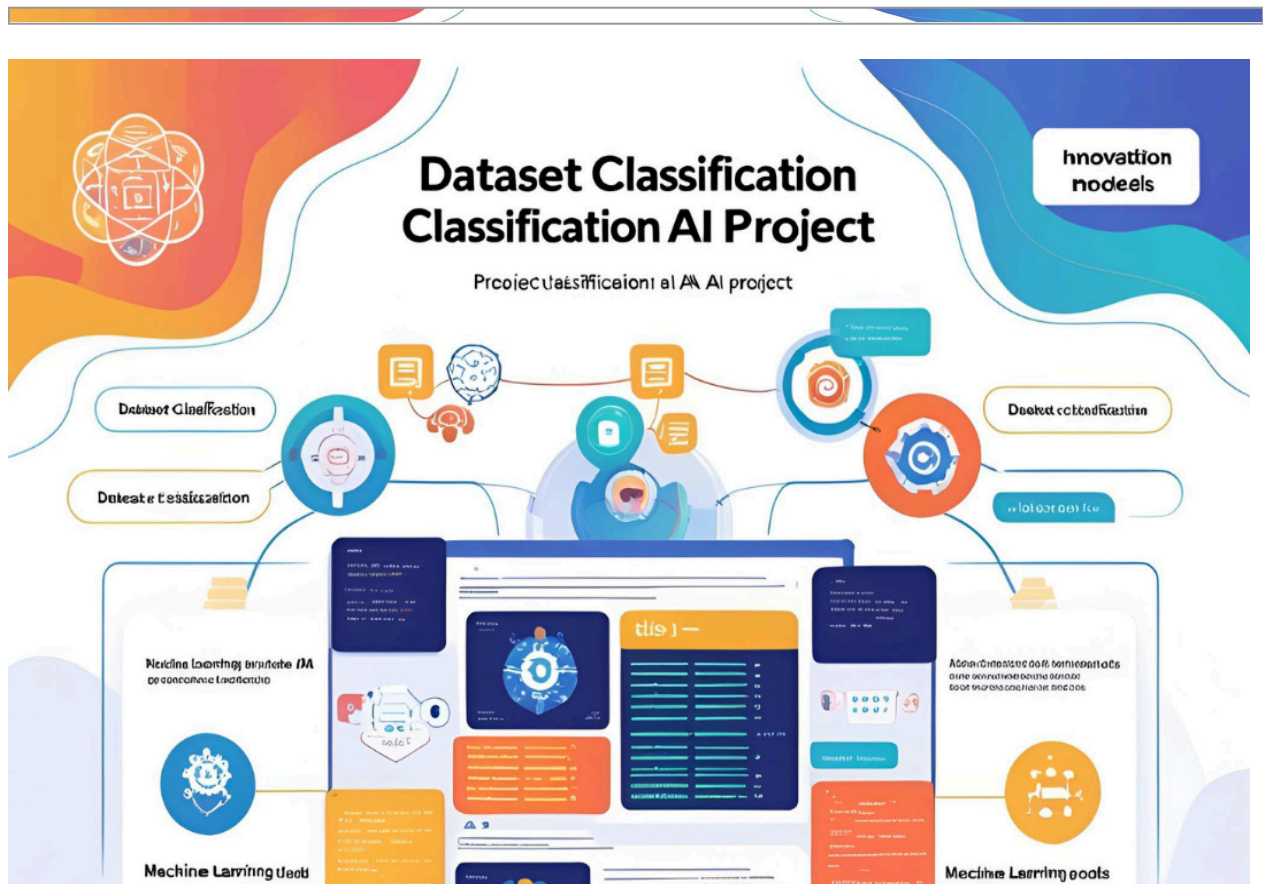




Dataset Classification Project

29.05.2025

RAJ PRATEEK PRAJAPATI

Dataset Classification Project - Environment Setup and Execution

This document outlines the complete environment setup and execution workflow for the *Dataset Classification machine learning* project using Python and Jupyter Notebook in Visual Studio Code.

It includes step-by-step instructions for creating a virtual environment, installing essential Python libraries, and registering the environment as a Jupyter kernel for seamless notebook development.

The guide ensures a reproducible and isolated setup, making it easy to manage dependencies and maintain consistency across machines. This setup forms the foundation for building and running a robust dataset classification model using classical machine learning algorithms.

Step 1: Install Required Libraries

- Open the VS Code terminal and run the following command to install essential Python libraries
- Run: **`pip install numpy pandas matplotlib scikit-learn ipykernel`**

These packages are essential for data processing, visualization, and modeling.

Step 2: Activate Virtual Environment

- If a virtual environment has already been created, activate it using:
`.\env\Scripts\activate # for Windows`

This isolates your project's dependencies.

Step 3: Install Jupyter Kernel Support

- To allow the virtual environment to function as a kernel inside Jupyter Notebook.
- Run: **`pip install ipykernel`**

Step 4: Register the Environment as a Jupyter Kernel

- Register the environment so it appears as a selectable kernel in Jupyter:
- Run: **`python -m ipykernel install --user --name dataset-cls-env --display-name "Python (Dataset Cls)"`**

Step 5: Switch Kernel in Jupyter Notebook

- Open the notebook 'Data Classification.ipynb', click the current kernel name in the top-right, and select: **Python (Dataset Cls)**

Step 6: Execute the Code

- Begin writing and running your ML code within the notebook. Test your setup with basic imports and data loading using pandas, numpy, matplotlib, etc.

Code Used

```
import sys

# Check the current Python version
print('Python: {}'.format(sys.version))

import scipy
# Print the version of SciPy
print('Scipy: {}'.format(scipy.__version__))

import numpy
# Print the version of NumPy
print('Numpy: {}'.format(numpy.__version__))

import matplotlib
# Print the version of Matplotlib
print('Matplotlib: {}'.format(matplotlib.__version__))

import pandas
# Print the version of Pandas
print('Pandas: {}'.format(pandas.__version__))

import sklearn
# Print the version of scikit-learn
print('Sklearn: {}'.format(sklearn.__version__))

import pandas
from pandas import read_csv
from pandas.plotting import scatter_matrix
from matplotlib import pyplot
from sklearn.model_selection import train_test_split
```

```
from sklearn.model_selection import cross_val_predict
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsRegressor
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn import model_selection
from sklearn.ensemble import VotingClassifier

# loading the dataset
from tkinter.font import names

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv"
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
dataset = pandas.read_csv(url, names=names)

# dimensions of the dataset
print(dataset.shape)

# take a peek at the dataset
print(dataset.head(20))

# Statistics summary
print(dataset.describe())

# class distribution
print(dataset.groupby('class').size())

# univariate plots- box and whisker plots
dataset.plot(kind='box', subplots=True, layout=(2,2), sharex=False, sharey=False)
pyplot.show()

# histograms of the variables
dataset.hist()
pyplot.show()

# multivariate plots
scatter_matrix(dataset)
pyplot.show()
```

```
# creating a validation dataset
# splitting the dataset into train and test sets
array = dataset.values
X = array[:,0:4] # features
Y = array[:,4] # target variable
X_train, X_validation, Y_train, Y_validation = train_test_split(X, Y, test_size=0.20,
random_state=1)

# We will Train this model on six different models and compare their performance
# Logistic Regression
# Linear Discriminant Analysis
# K-Nearest Neighbors
# Classification and Regression Trees
# Gaussian Naive Bayes
# Support Vector Machines

# Building the models
models = []
models.append(('LR', LogisticRegression(solver='liblinear', multi_class='ovr'))) # Logistic
Regression
models.append(('LDA', LinearDiscriminantAnalysis())) # Linear Discriminant Analysis
models.append(('KNN', KNeighborsRegressor())) # K-Nearest Neighbors
models.append(('CART', DecisionTreeClassifier())) # Classification and Regression Trees
models.append(('NB', GaussianNB())) # Gaussian Naive Bayes
models.append(('SVM', SVC(gamma='auto'))) # Support Vector Machines

# evaluate each model in turn
from sklearn.model_selection import cross_val_score

results = []
names = []
for name, model in models:
    kfold = StratifiedKFold(n_splits=10, random_state=1, shuffle=True)
    cv_results = cross_val_score(model, X_train, Y_train, cv=kfold, scoring='accuracy')
    results.append(cv_results)
    names.append(name)
    print('%s: %f (%f)' % (name, cv_results.mean(), cv_results.std()))

# Compare the models
pyplot.boxplot(results, labels=names)
pyplot.title('Algorithm Comparison')
pyplot.show()

# Make predictions on SVM model
model = SVC(gamma='auto')
```

```
model.fit(X_train, Y_train)
predictions = model.predict(X_validation)

# Evaluate predictions
print(accuracy_score(Y_validation, predictions))
print(confusion_matrix(Y_validation, predictions))
print(classification_report(Y_validation, predictions))
```

Result :

0.9666666666666667

[[11 0 0]

[0 12 1]

[0 0 6]]

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Iris-setosa	1.00	1.00	1.00	11
Iris-versicolor	1.00	0.92	0.96	13
Iris-virginica	0.86	1.00	0.92	6

accuracy			0.97	30
macro avg	0.95	0.97	0.96	30
weighted avg	0.97	0.97	0.97	30